

Unbalanced Data

ST 4035

Dealing with unbalanced data

```
# setting working directory

# install.packages("ROSE")
library(ROSE)
library(caret) # for the confusion matrix
```

Reading hacide data.

```
setwd("C:/Users/samee/My Drive/Teaching/ST 4035 - Data Science/2025")
hacide.train = read.csv("hacide.train.csv")
hacide.test = read.csv("hacide.test.csv")
hacide.test$cls = factor(hacide.test$cls)
hacide.train$cls = factor(hacide.train$cls)

str(hacide.train)
```

```
'data.frame':  1000 obs. of  3 variables:
 $ cls: Factor w/ 2 levels "0","1": 1 1 1 1 1 1 1 1 1 1 ...
 $ x1 : num  0.2008 0.0166 0.2287 0.1264 0.6008 ...
 $ x2 : num  0.678 1.5766 -0.5595 -0.0938 -0.2984 ...
```

```
str(hacide.test)
```

```
'data.frame':  250 obs. of  3 variables:
 $ cls: Factor w/ 2 levels "0","1": 1 1 1 1 1 1 1 1 1 1 ...
 $ x1 : num  0.0556 -0.7453 -0.1849 -0.98 0.1063 ...
 $ x2 : num  2.0987 -2.849 0.3807 0.0189 0.9021 ...
```

What is the severity of the class imbalance?

```
table(hacide.train$cls)
```

```
 0   1
980 20
```

```
prop.table(table(hacide.train$cls))
```

```
 0   1
0.98 0.02
```

Fitting a binary logistic

```
LR_model = glm(hacide.train$cls~., family=binomial(link="logit"),
               data=hacide.train)
summary(LR_model)
```

Call:

```
glm(formula = hacide.train$cls ~ ., family = binomial(link = "logit"),
    data = hacide.train)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-6.0046	0.5903	-10.172	< 2e-16 ***
x1	-1.6424	0.3544	-4.635	3.57e-06 ***
x2	-1.5961	0.3080	-5.182	2.20e-07 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 196.08 on 999 degrees of freedom
Residual deviance: 118.20 on 997 degrees of freedom
AIC: 124.2

Number of Fisher Scoring iterations: 8

Getting predictions

```
LR_predicted_probs = predict.glm(LR_model, newdata = hacide.test, type = "response")
LR_predicted_classes = ifelse(LR_predicted_probs > 0.5,1,0)
table(LR_predicted_classes)
```

```
LR_predicted_classes
0
250
```

Accuracy

```
mean(hacide.test$cls == LR_predicted_classes)
```

```
[1] 0.98
```

```
## 98% - since the data set is unbalanced, we cannot trust this
```

Using Rose package

```
accuracy.meas(hacide.test$cls, LR_predicted_classes)
```

Call:

```
accuracy.meas(response = hacide.test$cls, predicted = LR_predicted_classes)
```

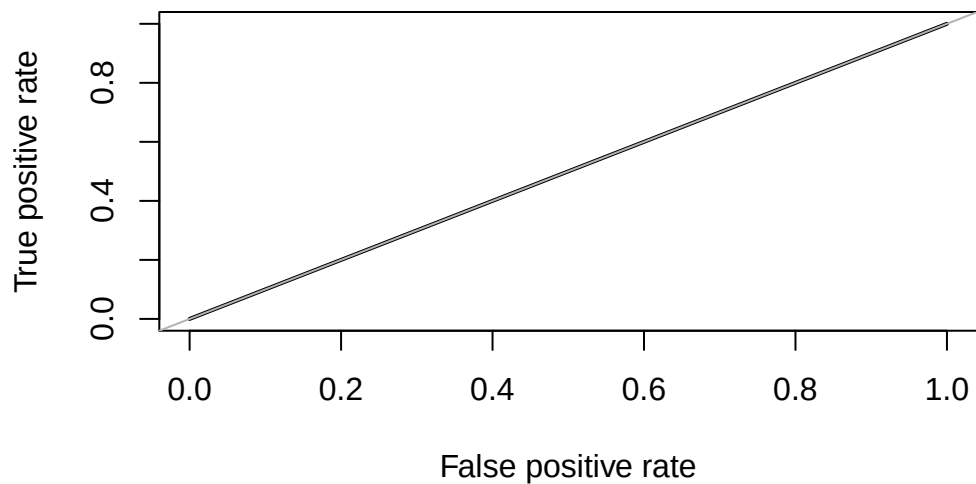
Examples are labelled as positive when predicted is greater than 0.5

```
precision: NaN
recall: 0.000
F: NaN
```

ROC curve

```
roc.curve(hacide.test$cls, LR_predicted_classes)
```

ROC curve



Area under the curve (AUC): 0.500

```
# confusion matrix  
contrasts(hacide.test$cls)
```

```
1  
0 0  
1 1
```

```
#contrasts(LR_predicted_classes)  
LR_predicted_classes = factor(LR_predicted_classes, levels=c(0,1))  
contrasts(LR_predicted_classes)
```

```
1  
0 0  
1 1
```

```
confusionMatrix(hacide.test$cls, LR_predicted_classes)
```

Confusion Matrix and Statistics

```

      Reference
Prediction 0  1
0 245    0
1   5    0

      Accuracy : 0.98
      95% CI : (0.9539, 0.9935)
No Information Rate : 1
P-Value [Acc > NIR] : 1.00000

      Kappa : 0

McNemar's Test P-Value : 0.07364

      Sensitivity : 0.98
      Specificity : NA
Pos Pred Value : NA
Neg Pred Value : NA
Prevalence : 1.00
Detection Rate : 0.98
Detection Prevalence : 0.98
Balanced Accuracy : NA

'Positive' Class : 0
```

oversampling to balance data

```
table(hacide.train$cls)
```

```

0  1
980 20
```

```
980*2
```

```
[1] 1960
```

```
data_oversampled = ovun.sample(cls ~ ., data = hacide.train, method="over", N=1960)
str(data_oversampled)
```

List of 3

```
$ Call : language ovun.sample(formula = cls ~ ., data = hacide.train, method = "over", N =
$ method: chr "over"
$ data : 'data.frame': 1960 obs. of 3 variables:
..$ cls: Factor w/ 2 levels "0","1": 1 1 1 1 1 1 1 1 1 1 ...
..$ x1 : num [1:1960] 0.2008 0.0166 0.2287 0.1264 0.6008 ...
..$ x2 : num [1:1960] 0.678 1.5766 -0.5595 -0.0938 -0.2984 ...
- attr(*, "class")= chr "ovun.sample"
```

```
table(data_oversampled$data$cls)
```

```
0 1
980 980
```

Ex: Fit a LR model for the oversampled dataset

Undersampling

```
data_underdsampled = ovun.sample(cls~., data = hacide.train, method = "under", N=40, seed = 1)
table(data_underdsampled$data$cls)
```

```
0 1
20 20
```

Ex: Fit a LR model for the undersampled dataset

SMOTE

```
#install.packages("smotefamily")
library(smotefamily)

data_SMOTE = SMOTE(hacide.train[,-1], hacide.train$cls)

table(data_SMOTE$data$class)
```

```
  0    1
980 980
```

```
str(data_SMOTE$data)
```

```
'data.frame':  1960 obs. of  3 variables:
 $ x1   : num  1.989 -2.164 -0.742 1.734 1.022 ...
 $ x2   : num  -1.108 -0.152 -1.974 -1.667 -1.802 ...
 $ class: chr   "1" "1" "1" "1" ...
```

```
data_SMOTE$data$class = factor(data_SMOTE$data$class)
```

Fit a LR model for the SMOTE dataset

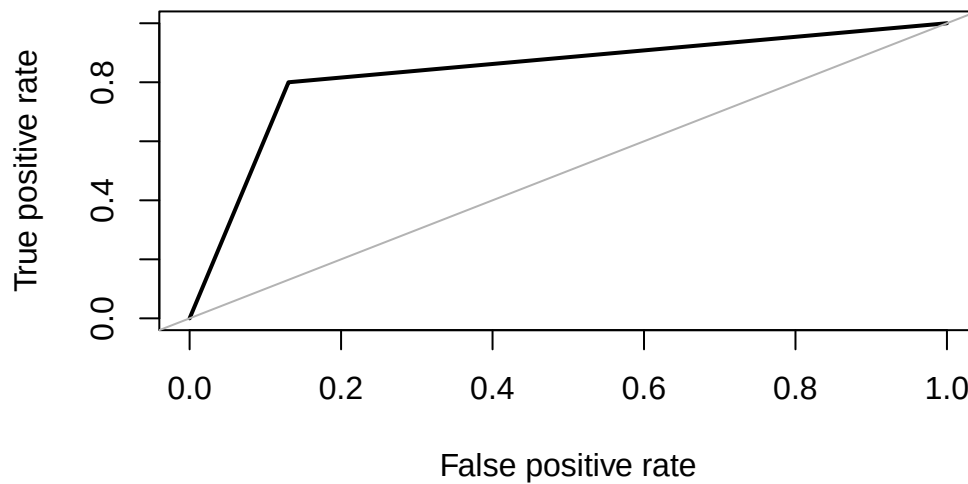
```
LR_model = glm(data_SMOTE$data$class~., family = binomial(link = 'logit'), data=data_SMOTE$d
LR_predicted_probs = predict.glm(LR_model, newdata = hacide.test, type='response')
LR_predicted_classes = ifelse(LR_predicted_probs > 0.5, 1,0)
table(LR_predicted_classes)
```

```
LR_predicted_classes
  0    1
214  36
```

Using ROSE package

```
roc.curve(hacide.test$cls, LR_predicted_classes)
```

ROC curve



Area under the curve (AUC): 0.835

Using Rose package

```
accuracy.meas(hacide.test$cls, LR_predicted_classes)
```

Call:

```
accuracy.meas(response = hacide.test$cls, predicted = LR_predicted_classes)
```

Examples are labelled as positive when predicted is greater than 0.5

precision: 0.111

recall: 0.800

F: 0.098